TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA CÔNG NGHỆ THÔNG TIN
***

CỘNG HÒA XÃ HỘI CHỦ NGHĨA VIỆT NAM
Độc lập - Tự do - Hạnh phúc
***

# ĐỀ CƯƠNG CHI TIẾT ĐỒ ÁN TỐT NGHIỆP

**Tên đề tài:** Xây dựng hệ thống gợi ý bài tập lập trình cá nhân hóa dựa trên phân tích lỗi mã nguồn.

**Sinh viên thực hiện:** gồm 01 sinh viên thực hiện

- **Họ và tên:** Trần Quốc
- **MSSV:** 102220335
- **Lớp SH:** 22T_Nhat2

**Tên người hướng dẫn:** TS. Phạm Minh Tuấn

## 1. Lý do chọn đề tài

Trong các chương trình đào tạo công nghệ thông tin hiện nay, lập trình là kỹ năng cốt lõi mà sinh viên cần rèn luyện liên tục thông qua thực hành. Tuy nhiên, phần lớn các hệ thống chấm bài tự động hiện tại như HackerRank, Codeforces chỉ đưa ra phản hồi dạng đúng/sai mà không giải thích rõ lý do sai, khiến sinh viên không biết mình đang mắc lỗi gì và cần cải thiện kỹ năng nào.

Bên cạnh đó, lộ trình bài tập thường được sắp xếp cố định và giống nhau cho toàn bộ lớp học, không phân biệt năng lực hay điểm yếu riêng của từng sinh viên. Điều này dẫn đến tình trạng: sinh viên giỏi cảm thấy nhàm chán vì bài quá dễ, trong khi sinh viên yếu lại bị bỏ lại phía sau vì không được hỗ trợ đúng chỗ.

Sự phát triển của các mô hình AI như CodeBERT (Microsoft) trong phân tích mã nguồn và Deep Knowledge Tracing trong theo dõi tiến trình học tập đã mở ra khả năng xây dựng một hệ thống thông minh hơn: tự động nhận diện loại lỗi trong code, đưa ra gợi ý sửa cụ thể, và cá nhân hóa lộ trình bài tập cho từng sinh viên dựa trên điểm yếu thực tế của họ.

Xuất phát từ những vấn đề thực tiễn trên, đề tài "Xây dựng hệ thống gợi ý bài tập lập trình cá nhân hóa dựa trên phân tích lỗi mã nguồn" được đề xuất nhằm hỗ trợ sinh viên học lập trình hiệu quả hơn, không chỉ biết mình sai mà còn hiểu sai ở đâu, sửa thế nào, và cần luyện thêm bài gì tiếp theo.

## 2. Mục tiêu và nhiệm vụ đề tài

- *Mục tiêu:* Xây dựng một website hỗ trợ luyện tập lập trình, trong đó khi người dùng nộp bài, hệ thống không chỉ chấm đúng/sai mà còn tự động phân tích lỗi mã nguồn, gợi ý hướng sửa, và đề xuất các bài tập phù hợp với điểm yếu cá nhân của từng người dùng.
- *Nhiệm vụ cụ thể:*
    + *Xây dựng môi trường làm bài trực tuyến:* Cho phép người dùng viết và chạy code trực tiếp trên trình soạn thảo tích hợp (hỗ trợ Python, Java, C++), sau đó nộp bài để hệ thống chấm điểm thông qua Judge0 API.

+ *Phân tích lỗi mã nguồn tự động:* Khi mã nguồn được hệ thống phát hiện lỗi sai, mô hình CodeBERT phân tích đoạn code và phân loại lỗi vào một trong các nhóm: lỗi logic, lỗi điều kiện vòng lặp, lỗi runtime (null, index out of range...), lỗi cú pháp.
+ *Gợi ý hướng sửa cụ thể:* Dựa trên loại lỗi được phân loại, hệ thống hiển thị gợi ý hướng sửa phù hợp, không đưa đáp án thẳng mà hướng dẫn người dùng tự suy nghĩ và tìm ra giải pháp.
+ *Gợi ý bài tập cá nhân hóa:* Dựa trên loại lỗi vừa phát hiện và bản đồ kỹ năng hiện tại, hệ thống đề xuất top 5 bài tập phù hợp nhất để người dùng luyện thêm, ưu tiên các kỹ năng đang yếu nhất.
+ *Xây dựng bản đồ kỹ năng cá nhân:* Sau mỗi lần nộp bài, mô hình Deep Knowledge Tracing cập nhật mức độ thành thạo của từng kỹ năng lập trình (vòng lặp, đệ quy, mảng, hàm...) và hiển thị dưới dạng radar chart trực quan.

**3. Đối tượng và phạm vi đề tài**

- *Đối tượng nghiên cứu:*
  + Mô hình CodeBERT trong bài toán phân loại lỗi mã nguồn.
  + Mô hình Deep Knowledge Tracing (DKT) dựa trên LSTM trong theo dõi tiến trình học tập.
  + Dataset: IBM Project CodeNet (CodeBERT), ASSISTments 2009 (DKT)
  + Thuật toán gợi ý bài tập dựa trên bản đồ kỹ năng cá nhân.
- *Đối tượng sử dụng:* Các đối tượng tham gia luyện DSA, sinh viên IT, …
- *Phạm vi thực hiện:*
  + Chức năng hệ thống:
    - Đăng ký, đăng nhập và quản lý tài khoản người dùng
    - Viết và chạy code trực tiếp trên trình soạn thảo code được tích hợp sẵn
    - Chấm bài tự động và trả kết quả đúng/sai theo từng test case
    - Phân tích lỗi mã nguồn và phân loại theo nhóm lỗi cụ thể
    - Gợi ý hướng sửa phù hợp với từng loại lỗi
    - Gợi ý top 5 bài tập cá nhân hóa theo điểm yếu
    - Hiển thị bản đồ kỹ năng cập nhật theo thời gian thực
  + Ngôn ngữ lập trình hỗ trợ: Python, Java, C/C++
  + Nền tảng triển khai: Ứng dụng Web

**4. Phương pháp thực hiện, công nghệ sử dụng**

- *Phương pháp thực hiện:*
  + Nghiên cứu tài liệu: Tìm hiểu các công trình liên quan đến phân tích lỗi mã nguồn bằng CodeBERT, mô hình Deep Knowledge Tracing trong giáo dục, và các hệ thống gợi ý bài tập lập trình đã được công bố.
  + Phương pháp thực nghiệm: Fine-tune CodeBERT trên tập dữ liệu code lỗi được gán nhãn theo loại lỗi. Fine-tune mô hình DKT trên dataset ASSISTments 2009, thông qua thư viện pyKT (Python). Đánh giá độ chính xác phân loại lỗi và chất lượng gợi ý bài tập.
- *Công nghệ sử dụng:*

| **Thành phần** | **Công nghệ sử dụng** |
|---|---|

| | |
|---|---|
| Frontend | ReactJS + Vite |
| Backend | FastAPI |
| Hệ thống chấm bài | Judge0 |
| Hệ thống phân tích lỗi | CodeBERT |
| Theo dõi kĩ năng | Deep Knowledge Tracing (LSTM) – pyKT |
| Cơ sở dữ liệu | PostgreSQL |
| Deployment | Docker, Vercel, Railway |

## 5. Dự kiến kết quả

### a. Sản phẩm hệ thống

- Hệ thống website gợi ý bài tập lập trình cá nhân hóa dựa trên phân tích lỗi mã nguồn, hỗ trợ ba ngôn ngữ lập trình Python, Java và C/C++ với đầy đủ các chức năng:
    + *Quản lý tài khoản:* Đăng ký, đăng nhập, xem hồ sơ cá nhân và lịch sử làm bài.
    + *Môi trường làm bài:* Trình soạn thảo code tích hợp (Monaco Editor), cho phép viết và chạy thử code trực tiếp trên trình duyệt.
    + *Chấm bài tự động:* Tích hợp hệ thống chấm bài tự động Judge0, trả kết quả đúng/sai theo từng test case kèm thời gian và bộ nhớ sử dụng.
    + *Phân tích lỗi mã nguồn:* Tự động phân loại lỗi vào bốn nhóm (Logic Error, Loop & Condition Error, Memory & Reference Error, Recursion Error) và hiển thị gợi ý hướng sửa phù hợp.
    + *Bản đồ kỹ năng cá nhân:* Radar chart cập nhật theo thời gian thực sau mỗi lần nộp bài, phản ánh mức độ thành thạo từng kỹ năng (vòng lặp, đệ quy, mảng, hàm, v.v.).
    + *Gợi ý bài tập cá nhân hóa:* Đề xuất top 5 bài tập phù hợp nhất dựa trên bản đồ kỹ năng hiện tại, ưu tiên các kỹ năng đang yếu nhất.

### b. Mô hình phân loại lỗi mã nguồn (CodeBERT)

- Mô hình CodeBERT được fine-tune trên tập dữ liệu code lỗi được gán nhãn theo bốn nhóm lỗi (Logic Error, Loop & Condition Error, Memory & Reference Error, Recursion Error). Dự kiến kết quả đánh giá trên tập test:

| Metric | Mục tiêu |
|---|---|
| Accuracy | ≥ 80% |
| Macro F1-score | ≥ 0.78 |
| Precision (trung bình các lớp) | ≥ 0.78 |
| Recall (trung bình các lớp) | ≥ 0.78 |

### c. Mô hình theo dõi kỹ năng (Deep Knowledge Tracing)

- Mô hình DKT dựa trên LSTM được huấn luyện trên lịch sử làm bài của người dùng, dự đoán xác suất người dùng trả lời đúng bài tập tiếp theo dựa trên chuỗi kết quả trước đó. Dự kiến kết quả:

| Metric | Mục tiêu |
|---|---|
| AUC-ROC | ≥ 0.80 |
| RMSE | ≤ 0.40 |

- Bản đồ kỹ năng cá nhân được cập nhật liên tục sau mỗi lần nộp bài, phản ánh đúng xu hướng tiến bộ (hoặc suy giảm) của người dùng theo thời gian.

**d. Hệ thống gợi ý bài tập cá nhân hóa**

- Thuật toán gợi ý kết hợp kết quả phân loại lỗi từ CodeBERT và bản đồ kỹ năng từ DKT để đề xuất top 5 bài tập phù hợp nhất. Dự kiến kết quả đánh giá:

| Metric | Mục tiêu |
|---|---|
| Precision@5 | ≥ 0.70 |
| Hit Rate@5 | ≥ 0.80 |

- Trong đó Precision@5 đo tỉ lệ bài tập gợi ý thực sự phù hợp với kỹ năng cần cải thiện, và Hit Rate@5 đo tỉ lệ trường hợp trong top 5 gợi ý có ít nhất một bài tập mà người dùng thực sự cần luyện tập.

**6. Ý nghĩa thực tiễn và khoa học**

- *Ý nghĩa thực tiễn:*
  + Các nền tảng luyện lập trình phổ biến hiện nay như LeetCode, Codeforces hay HackerRank đều dừng lại ở một giới hạn chung: chỉ thông báo kết quả đúng hoặc sai, không giải thích tại sao sai và cần sửa gì. Điều này khiến người học – đặc biệt là sinh viên mới tiếp cận lập trình – rơi vào trạng thái lúng túng, không biết bắt đầu gỡ lỗi từ đâu.
  + Hệ thống được đề xuất trong đề tài này giải quyết trực tiếp vấn đề đó. Thay vì chỉ trả về đúng/sai, hệ thống tự động nhận diện loại lỗi trong mã nguồn, đưa ra gợi ý hướng sửa phù hợp và dẫn người dùng đến đúng bài tập cần luyện tập tiếp theo.
- *Ý nghĩa khoa học:*
  + Thứ nhất, đề tài cung cấp kết quả thực nghiệm về khả năng ứng dụng của CodeBERT trong bài toán phân loại lỗi mã nguồn theo ngữ nghĩa – một hướng ứng dụng còn ít được khảo sát so với các bài toán truyền thống như tìm kiếm code hay sinh tài liệu. Kết quả fine-tune trên tập dữ liệu lỗi thực tế góp phần làm rõ giới hạn và tiềm năng của mô hình ngôn ngữ lớn trong phân tích chất lượng code.
  + Thứ hai, đề tài mở rộng phạm vi ứng dụng của Deep Knowledge Tracing sang môi trường học lập trình – vốn có đặc thù khác biệt so với các môn học truyền thống (câu hỏi trắc nghiệm, toán học) mà DKT thường được áp dụng.

**7. Dự kiến nội dung ĐATN**

- **Chương 1:** Tổng quan về đề tài (Giới thiệu bài toán, lý do chọn đề tài, mục tiêu, phạm vi và giải pháp đề xuất).

- **Chương 2:** Cơ sở lý thuyết (Kiến thức về hệ thống chấm bài tự động Online Judge, mô hình CodeBERT trong phân tích mã nguồn, mô hình Deep Knowledge Tracing, thuật toán gợi ý bài tập cá nhân hóa).
- **Chương 3:** Phân tích và thiết kế hệ thống (Kiến trúc tổng thể hệ thống, thiết kế cơ sở dữ liệu, thiết kế luồng xử lý submit → phân tích lỗi → gợi ý bài tập, quy trình huấn luyện mô hình AI).
- **Chương 4:** Triển khai và đánh giá kết quả (Giao diện website, kết quả phân loại lỗi mã nguồn của CodeBERT, biểu đồ bản đồ kỹ năng cá nhân, đánh giá chất lượng gợi ý bài tập).
- **Chương 5:** Kết luận và hướng phát triển.

**8. Dự kiến tiến độ**

| **Tuần** | **Giai đoạn** | **Nội dung công việc chi tiết** |
|---|---|---|
| 1 | Nghiên cứu | Đọc tài liệu về CodeBERT, Deep Knowledge Tracing, Judge0 API. Phân tích các hệ thống tương tự (LeetCode, Codeforces). Xác định yêu cầu chức năng và phi chức năng |
| 2 | Thiết kế | Thiết kế kiến trúc tổng thể hệ thống, thiết kế cơ sở dữ liệu, vẽ wireframe giao diện các màn hình chính, thiết kế luồng xử lý submit → phân tích → gợi ý |
| 3 | Backend | Xây dựng Backend: API đăng ký/đăng nhập, quản lý bài tập, tích hợp Judge0 API chấm bài tự động, xử lý kết quả test case |
| 4 | Frontend | Xây dựng Frontend: trang danh sách bài tập, trình soạn thảo code tích hợp (Monaco Editor), màn hình nộp bài và hiển thị kết quả đúng/sai |
| 5 | AI – CodeBERT | Thu thập và gán nhãn tập dữ liệu code lỗi (lỗi logic, vòng lặp, runtime, cú pháp). Fine-tune mô hình CodeBERT. Đánh giá độ chính xác phân loại lỗi |
| 6 | Tích hợp AI vào hệ thống | Tích hợp CodeBERT vào Backend. Xây dựng bảng ánh xạ loại lỗi → gợi ý hướng sửa. Hiển thị phân tích lỗi và gợi ý sửa lên giao diện người dùng |
| 7 | AI – DKT và xây dựng thuật toán gợi ý bài tập | Xây dựng và huấn luyện mô hình Deep Knowledge Tracing thông qua thư viện python (pyKT) trên dataset ASSISTments 2009. Xây dựng bản đồ kỹ năng cá nhân. Hiển thị radar chart trực quan trên giao diện |

| | | |
|---|---|---|
| | | Xây dựng thuật toán gợi ý bài tập ưu tiên theo kỹ năng yếu nhất. Tích hợp toàn bộ luồng hệ thống: submit → lỗi → bản đồ kỹ năng → gợi ý bài |
| 8 | Kiểm thử và deploy | Kiểm thử từng module (unit test), kiểm thử tích hợp toàn hệ thống, kiểm thử với người dùng thực tế, phát hiện và sửa lỗi, tối ưu hiệu năng. Triển khai deploy website. |
| 9 | Hoàn thiện | Viết báo cáo đồ án đầy đủ theo cấu trúc 5 chương. Chuẩn bị slide trình bày. Review lại toàn bộ hệ thống và báo cáo. |

*Đà Nẵng, ngày 28 tháng 3 năm 2026*

| XÁC NHẬN CỦA NGƯỜI HƯỚNG DẪN | SINH VIÊN THỰC HIỆN |
|---|---|
| (*Ký và ghi rõ họ tên*) | (*Ký và ghi rõ họ tên*) |
| TS. Phạm Minh Tuấn | Trần Quốc |